Limpieza de datos de un proyecto de RFM (Recencia, Frecuencia, Valor Monetario). Proyecto de portafolio de Analista de Datos Jr

Parte II Análisis de RFM (Recencia, Frecuencia, Valor Monetario)

Librerías que se usaran en este proyecto

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import json
import math
#Para hacer limpieza de errores tipograficos a gran escala
import unicodedata
import re
#Para el calculo de los productos que se venden en pareja
from itertools import combinations
from collections import Counter
```

Estas son las numerables librerías que se estaría usando en este proyecto las cuales serán aclaradas en otra presentación, pero por ahora solo se estarían usando:

- **Pandas:** Utilizada para la manipulación y análisis de datos. Crea y gestiona DataFrames, carga archivos (CSV, Excel), realiza operaciones de limpieza, filtrado, agrupación y unión de datos.
- **Numpy:** Específicamente para la función np.select, que se usa para asignar los segmentos cualitativos de clientes (Champions, Loyal Customers, etc.) basados en múltiples condiciones.
- **Matplotlib:** Es la librería fundamental para la creación de gráficos estáticos, aunque en este código, muchas visualizaciones se delegan a Seaborn, Matplotlib se usa implícitamente por Seaborn y para la configuración general de las figuras y subgráficos.
- **Seaborn:** Se utiliza para crear visualizaciones estadísticas más atractivas y complejas a partir de los DataFrames de Pandas, como boxplots, histogramas, barplots, lineplots y heatmaps, para entender distribuciones y patrones de datos.

```
#Creacion de una nueva columna con valores totales
df_rfm['total'] = df_rfm['quantity'] * df_rfm['price']
```

En esta celda se crea una nueva columna en el dataframe df_rfm para saber el totol de la operación es decir se multiplica la cantidad del producto comprado con el precio del mismo en la nueva columna total

```
#Fecha de referencia
snapshot_date = df_rfm['invoicedate'].max() + pd.Timedelta(days=1)
```

Ahora se busca la ultima compra (fecha mas reciente) con `.max()` y se le agrega un día para este caso con el uso de la función `pd.Timedelta(days=1)` ubicando el dia después de la última transacción registrada. Esto es una práctica común en el análisis RFM para asegurar que la recencia de los clientes se calcule correctamente (es decir, cuántos días han pasado desde su última compra hasta el día después de la última compra en el dataset). Todo esto se almacena en una nueva variable llamada `snapshot_date`

```
#Agrupacion por customer_id
rfm = df_rfm.groupby('customer_id').agg({
    'invoicedate' : lambda x: (snapshot_date - x.max()).days,
    'invoice' : 'nunique',
    'total' : 'sum'
})
```

Ya aquí se agrupan los clientes por ID es decir si un cliente realizo 5 operaciones se agrupan las transacciones tomando en cuenta la columna “customer_id” y realizamos los siguientes procedimientos aplicando el método `.agg({})` para analizar múltiples operaciones de agregación en las columnas deseadas para cada grupo de clientes. Dentro de estas operaciones se crea un diccionario donde cada clave es el nombre de una columna original y su valor es la función de agregación o una lista de funciones a aplicar.

`'invoicedate' : lambda x: (snapshot_date - x.max()).days`

Aquí iniciamos con el cálculo la métrica de Recencia donde:

- `'invoicedate'` es la columna a la que se aplica la agregación.
- `lambda x: (snapshot_date - x.max()).days` es una función anónima (lambda) que se ejecuta para la columna `invoicedate` dentro de cada grupo de clientes:
  - x representa la serie de `invoicedate` para un cliente específico.

        - `x.max()` encuentra la fecha más reciente de compra de ese cliente (su última compra).
        - `snapshot_date - x.max()` calcula la diferencia de tiempo entre la `snapshot_date` (la fecha de referencia que se definio anteriormente como el día después de la última transacción global) y la última compra del cliente. Esto da un objeto `Timedelta`.
        - `.days` extrae el número de días de esa diferencia de tiempo. El resultado es la recencia del cliente (cuántos días han pasado desde su última compra hasta la fecha de referencia).

- `'invoice' : 'nunique'`
    - Con esta línea calcula la métrica de Frecuencia.
    - `'invoice'` es la columna que contiene los números de factura.
    - `'nunique'` es una cadena que indica la función de agregación la cual cuenta el número de valores únicos en la columna `invoice` para cada cliente. Esto representa la cantidad de pedidos o transacciones únicas que ha realizado cada cliente.

- `'total' : 'sum'`
    - Esta línea calcula la métrica de Monetario.
    - `'total'` es la columna que contiene el valor total de cada transacción (cantidad * precio) la que se creó anteriormente.
    - `'sum'` es una cadena que indica la función de agregación: suma todos los valores de la columna `total` para cada cliente. Esto representa el gasto total de cada cliente.

```
#Renombrando columnas
rfm.rename(columns={
    'invoicedate' : 'recency',
    'invoice' : 'frequency',
    'total' : 'monetary'
}, inplace=True)
```

Aquí solo se renombra las columnas con `.rename()` para su fácil interpretación donde la columna “invoicedate la renombramos por “recency” y así mantenemos los nombres técnicos para el análisis y así con las otras 2 columnas restantes y modificamos en dataframe de manera definitiva con `inplace=True`

```
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10))
cols = ['recency', 'frequency', 'monetary']

for i, col in enumerate(cols):
  #Histograma con curva de densidad
  sns.histplot(rfm[col], bins=50, kde=True, ax=axes[0, i],
color='skyblue')
  axes[0, i].set_title(f'Distribution of {col.capitalize()}',
fontweight='bold')

  #Boxplots para detectar outliers
  sns.boxplot(y=rfm[col], ax=axes[1, i], color='lightgreen')
  axes[1, i].set_title(f'Boxplot of {col.capitalize()}',
fontweight='bold')

plt.tight_layout()
plt.show()
```

Primero se configura el lienzo donde se crea una figura con `plt.subplots` que tendrá 6 gráficos (2 filas por 3 columnas) para organizar las visualizaciones de cada métrica. Seguido con un bucle for se Itera por métricas y recorre las columnas `recency, frequency y monetary` las cuales están almacenadas en la variable `cols.`

Aquí creamos 2 tipos de gráficos y empezamos con el Histograma y KDE para cada métrica, crea un histograma (`sns.histplot`) en la fila superior, mostrando la distribución de los datos y una curva de densidad (KDE).

Luego continuamos con los Boxplot: En la fila inferior, genera un diagrama de caja (`sns.boxplot`), que es excelente para identificar valores atípicos (outliers) y la dispersión de los datos.

De ultimo, `plt.tight_layout()` ajusta los gráficos para que no se superpongan y `plt.show()` los muestra.

Como Conclusión se llegó a:

Distribution of Recency:

- Conclusiones: La Distribución esta sesgada hacia la izquierda (valores bajos), lo que indica una base de clientes mayoritariamente activa recientemente. El boxplot muestra que el 50% de los clientes compraron hace menos de 100 días aproximadamente
- Recomendación: Los clientes que superan el umbral de los 300 días deben ser el objetivo de campañas de reactivación inmediatas antes de que se consideren "perdidos" o lost custemers

Distribution of Frequency

- Conclusión: Tanto el histograma y el boxplot revelan una frecuencia muy baja para la masa crítica: la mayoría solo tienen 1 o 2 transacciones. Los "outliers" en el boxplot son los clientes verdaderamente fieles que compran recurrentemente
- Recomendación: El desafío del negocio es la "compra repetida". Se deben crear programas de lealtad que incentiven una segunda y tercera compra

Distribution of Monetary

- Conclusión: Existe una distribución extrema. Mientras el promedio general es bajo, unos pocos clientes generan ingresos masivos (outliers que superan los 500.000)
- Recomendación: Evaluar si existe la regla de Pareto. debido que posiblemente el 20% de los clientes den el 80% de los ingresos. Es ideal proteger estos clientes de alto valor monetario con atención personalizada

```
rfm['r_score'] = pd.qcut(rfm['recency'], q=5, labels=[5, 4, 3, 2,
1]).astype(int)
```

Aquí empezamos con la división para la asignación de puntuación para más adelante definir los clientes por categoría

  - Esta linea crea una nueva columna llamada `r_score` en el dataframe df_rfm (que contiene las métricas RFM). En esta columna se almacenará la puntuación de recencia para cada cliente.

- `pd.qcut(rfm['recency'], ...)`
  - `pd.qcut()` es una función de Pandas que se utiliza para dividir los datos en cuantiles (grupos con un número igual de observaciones).
  - `rfm['recency']`: Este es el input principal. Se refiere a la columna de recencia del dataframe df_rfm, que contiene el número de días desde la última compra de cada cliente.
  - `q=5`: Esto le dice a `qcut` que divida los datos de recencia en 5 cuantiles, creando 5 grupos de clientes. Cada grupo contendrá aproximadamente el mismo número de clientes.
  - `labels=[5, 4, 3, 2, 1]`: Esta es la parte clave para la lógica RFM de recencia.
    - Normalmente, `qcut` asignaría etiquetas numéricas consecutivas o los rangos de los cuantiles. Sin embargo, en RFM, una recencia menor (es decir, el cliente compró más recientemente) es mejor.
    - Al pasar `[5, 4, 3, 2, 1]`, se asigna la puntuación más alta (5) a los clientes con la recencia más baja (los que compraron más recientemente, que estarán en el primer cuantil), y la puntuación más baja (1) a los clientes con la recencia más alta (los que compraron hace más tiempo, en el último cuantil).

- `.astype(int)`:
    - Después de que `qcut` asigna las etiquetas (que pueden ser de tipo categórico o numérico dependiendo de los `labels` dados), `.astype(int)` convierte estas puntuaciones a tipo entero para facilitar operaciones posteriores o simplemente por claridad.

En pocas palabras esta línea toma a todos los clientes, los divide en 5 grupos iguales según qué tan recientemente compraron, y luego les asigna una puntuación del 1 al 5, donde 5 significa que compraron muy recientemente (lo cual es bueno) y 1 significa que compraron hace mucho tiempo (lo cual no es tan bueno para la recencia).

```
#Frecuencia y Monetario: Mas es mejor(Score 5)
rfm['f_score'] = pd.qcut(rfm['frequency'].rank(method='first'), q=5,
labels=[1, 2, 3, 4, 5]).astype(int)

rfm['m_score'] = pd.qcut(rfm['monetary'].rank(method='first'), q=5, labels=[1,
2, 3, 4, 5]).astype(int)
```

```
rfm['f_score'] = pd.qcut(rfm['frequency'].rank(method='first'), q=5,
labels=[1, 2, 3, 4, 5]).astype(int)
```

De nuevo, esta otra línea crea una nueva columna llamada `f_score` en el dataframe df_rfm. Aquí se almacenará la puntuación de frecuencia para cada cliente.

- `rfm['frequency'].rank(method='first')`:
    - `rfm['frequency']`: Esta es la columna de frecuencia de los clientes, que indica cuántos pedidos únicos ha realizado cada uno.
    - `.rank(method='first')`: La función `rank()` asigna un rango a cada valor en la columna de `frequency`.
        - Cuando hay valores empatados (varios clientes tienen la misma frecuencia, por ejemplo, muchos clientes han hecho 1 pedido), `method='first'` asegura que a cada cliente se le asigne un rango único, resolviendo los empates según el orden de aparición original del cliente en el dataframe.
        - Esto es necesario antes de usar `qcut` porque `qcut` funciona mejor con valores únicos o una distribución que no tenga muchos empates en los puntos de corte, asegurando que cada cliente caiga en un cuantil definido.
- `pd.qcut(..., q=5, labels=[1, 2, 3, 4, 5])`:
    - `pd.qcut()` nuevamente divide los datos (ahora los rangos de frecuencia) en 5 cuantiles, creando 5 grupos con aproximadamente el mismo número de clientes.
    - `q=5`: Divide en 5 grupos.

- `labels=[1, 2, 3, 4, 5]`: A diferencia de la recencia, en frecuencia, un número mayor de compras es mejor. Por lo tanto, los clientes con la frecuencia más baja (primer cuantil de rangos) obtendrán un 1, y los clientes con la frecuencia más alta (último cuantil de rangos) obtendrán un 5.

- `.astype(int)`:
  - Convierte las puntuaciones resultantes a tipo entero.

Es decir, esta línea asigna una puntuación de frecuencia del 1 al 5 a cada cliente. Primero, rankea a los clientes según su número de compras (resolviendo empates), luego los divide en 5 grupos iguales basándose en esos rangos, y finalmente les asigna una puntuación, donde un 5 indica una frecuencia de compra muy alta (lo cual es muy bueno) y un 1 una frecuencia baja (no tan bueno).

```
rfm['m_score'] = pd.qcut(rfm['monetary'].rank(method='first'), q=5,
labels=[1, 2, 3, 4, 5]).astype(int)
```

Similar a las líneas anteriores, esta instrucción crea una nueva columna llamada `m_score` en el dataframe df_rfm. Esta columna almacenará la puntuación monetaria para cada cliente.

- `rfm['monetary'].rank(method='first')`:
  - `rfm['monetary']`: Esta es la columna que contiene el valor monetario total de las compras de cada cliente.
  - `.rank(method='first')`: Al igual que con la frecuencia, la función `rank()` asigna un rango a cada valor en la columna `monetary`.
    - El uso de `method='first'` es crucial aquí para manejar los empates (es decir, cuando varios clientes tienen exactamente el mismo valor monetario). Asegura que a cada cliente se le asigne un rango único, lo cual es importante para que `qcut` pueda agruparlos de manera efectiva sin ambigüedades.
    - Esto prepara los datos para una división equitativa en cuantiles, incluso si hay clientes que han gastado la misma cantidad.

- `pd.qcut(..., q=5, labels=[1, 2, 3, 4, 5])`:
  - `pd.qcut()` nuevamente toma los rangos de los valores monetarios y los divide en 5 cuantiles, creando 5 grupos que contienen aproximadamente el mismo número de clientes.
  - `q=5`: Divide el conjunto de clientes en 5 grupos según sus rangos monetarios.
  - `labels=[1, 2, 3, 4, 5]`: Para el valor monetario, una cantidad mayor gastada es mejor. Por lo tanto:
    - Los clientes con el valor monetario más bajo (es decir, en el cuantil más bajo de rangos) reciben una puntuación de 1.
    - Los clientes con el valor monetario más alto (en el cuantil más alto de rangos) reciben una puntuación de 5.

- `.astype(int):`
    - Finalmente, esto convierte las puntuaciones resultantes en números enteros.

Es decir, esta otra línea asigna a cada cliente un puntaje monetario del 1 al 5. Primero, rankea a los clientes según el monto total que han gastado (manejando los empates), luego los divide en 5 grupos iguales basados en esos rangos, y asigna los puntajes de manera que un 5 indica un gasto muy alto (excelente) y un 1 indica un gasto bajo (no tan bueno).

```
#Score Total
rfm['rfm_score'] = rfm['r_score'] + rfm['f_score'] + rfm['m_score']

rfm['rfm_segment'] = rfm['r_score'].astype(str) + rfm['f_score'].astype(str) +
rfm['m_score'].astype(str)
```

Estas dos líneas de código son los pasos finales para consolidar las puntuaciones RFM individuales en una única métrica y un identificador de segmento:

- `rfm['rfm_score'] = rfm['r_score'] + rfm['f_score'] + rfm['m_score']:`

Esta línea calcula el puntaje RFM total para cada cliente. Simplemente suma las tres puntuaciones individuales que se calcularon anteriormente en `r_score` (recencia), `f_score` (frecuencia) y `m_score` (monetario). Cada cliente tendrá ahora un `rfm_score` que va desde un mínimo de 3 (1+1+1, para los clientes menos valiosos) hasta un máximo de 15 (5+5+5, para los clientes más valiosos o 'Champions'). Este puntaje numérico se usará para ordenar y categorizar a los clientes de forma rápida.

- `rfm['rfm_segment'] = rfm['r_score'].astype(str) + rfm['f_score'].astype(str) + rfm['m_score'].astype(str):`

Ahora esta línea crea una cadena de texto que representa el segmento RFM específico de cada cliente. Para cada puntuación (R, F, M):

- `rfm['r_score'].astype(str)`: Convierte el puntaje de recencia (que es un número entero) a una cadena de texto. Por ejemplo, un 5 se convierte en "5".
- Lo mismo ocurre con `rfm['f_score'].astype(str)` y `rfm['m_score'].astype(str)`.
- Luego, estas tres cadenas se concatenan (se unen una al lado de la otra). El operador + en Python, cuando se usa con cadenas, realiza la concatenación.

Cada cliente obtiene una cadena como "555" (si es un Champion) o "111" (si es un Lost Customer), o "342" (para un cliente con recencia media, buena frecuencia y bajo valor monetario). Esta cadena es un identificador único para el segmento RFM basado en las puntuaciones exactas de R, F y M de cada cliente.

```
conditions = [
    (rfm['rfm_score'] >= 13),
    (rfm['rfm_score'] >= 10) & (rfm['rfm_score'] < 13),
    (rfm['rfm_score'] >= 7) & (rfm['rfm_score'] < 10),
    (rfm['rfm_score'] >= 5) & (rfm['rfm_score'] < 7)
]
choices = ['Champions', 'Loyal Customers', 'Potential Loyalists', 'At Risk']
rfm['customer_segment'] = np.select(conditions, choices, default= 'Lost
Customers')
```

Esta celda es donde se traduce el `rfm_score` numérico en segmentos de clientes cualitativos y fáciles de interpretar. Así es como funciona:

- `conditions = [...]`:
  - Esto es una lista de condiciones lógicas que se aplican a la columna `rfm_score` del dataframe df_`rfm`.
  - Cada elemento de la lista es una Serie Booleana (True/False) que indica si un cliente cumple esa condición.
  - Por ejemplo:
    - `(rfm['rfm_score'] >= 13)`: Identifica a los clientes cuyo `rfm_score` es 13 o más alto.
    - `(rfm['rfm_score'] >= 10) & (rfm['rfm_score'] < 13)`: Identifica a los clientes cuyo `rfm_score` está entre 10 y 12 (inclusive).
    - Las condiciones están ordenadas de mayor a menor rfm_score, lo cual es importante para el funcionamiento de `np.select`.

- `choices = ['Champions', 'Loyal Customers', 'Potential Loyalists', 'At Risk']`:
  - Esta es una lista de los nombres de los segmentos que corresponden directamente a las `conditions` definidas anteriormente.
  - El primer `choice` ("Champions") se asignará si la primera `condition` es True, el segundo `choice` ("Loyal Customers") si la segunda `condition` es True, y así sucesivamente.

- `rfm['customer_segment'] = np.select(conditions, choices, default= 'Lost Customers')`:

- `rfm['customer_segment'] = ...`: Esto crea una nueva columna en el dataframe df_rfm que contendrá el nombre del segmento de cliente para cada entrada.
- `np.select()`: Esta es una función de NumPy que permite asignar valores basándose en múltiples condiciones. Trabaja de la siguiente manera:
  - Toma la lista `conditions`.
  - Toma la lista `choices`.
  - Para cada fila en el dataframe, `np.select` itera a través de las `conditions` en orden. Tan pronto como encuentra la primera condición que es True para esa fila, asigna el `choice` correspondiente a esa fila.
  - `default='Lost Customers'`: Este es un argumento que si una fila (un cliente) no cumple ninguna de las condiciones especificadas en la lista `conditions` (por ejemplo, si su `rfm_score` es menor que 5), entonces se le asigna el valor 'Lost Customers' (clientes perdidos) por defecto.

Esta celda toma el puntaje RFM numérico de cada cliente y, siguiendo una serie de reglas predefinidas, lo clasifica en un segmento de cliente cualitativo donde:

- Champions: clientes más valiosos y leales. Han comprado recientemente, lo hacen con frecuencia y gastan mucho.
- Loyal Customers: Clientes que compran con regularidad y gastan una buena cantidad, pero quizás no tan recientemente o en volúmenes tan altos como los Champions.
- Potential Loyalists: Clientes que han comprado recientemente y gastan una cantidad decente, pero su frecuencia aún no es alta. Tienen el potencial de convertirse en clientes leales.
- At Risk: Clientes que no han comprado en un tiempo considerable y/o han gastado poco. Podrían estar a punto de perderse si no se interviene.

```
segment_analysis = rfm.groupby('customer_segment').agg({
    'recency' : 'mean',
    'frequency' : 'mean',
    'monetary' : ['mean', 'count']
}).round(1)

#Modificacion de los titulos de las columnas
segment_analysis.columns = ['_'.join(col).strip() for col in
segment_analysis.columns.values]

display(rfm.head())
print("\n")
print("\033[1;4mAnalisis por Segmento\033[0m")
display(segment_analysis)
```

Esta celda de código es crucial para obtener un resumen estadístico de cada segmento de clientes, permitiéndote entender mejor sus características. Así es como funciona:

- `segment_analysis = rfm.groupby('customer_segment').agg({ ... }).round(1):`
    - `rfm.groupby('customer_segment')`: Esto agrupa el dataframe (que ahora tiene las puntuaciones RFM y los segmentos de clientes) por la columna `customer_segment`.
    - `.agg({ ... })`: Una vez agrupados los clientes por su segmento, se aplican funciones de agregación a las columnas especificadas:
        - `'recency': 'mean'`: Calcula el promedio de la recencia (días desde la última compra) para todos los clientes en cada segmento. Por ejemplo, dirá cuál es la recencia media de los 'Champions' frente a los 'Lost Customers'.
        - `'frequency': 'mean'`: Calcula la frecuencia promedio (número de pedidos únicos) para cada segmento.
        - `'monetary': ['mean', 'count']`: Para la columna monetaria, calcula dos cosas:
            - El promedio del gasto total (`'mean'`) para cada segmento.
            - El número de clientes (`'count'`) dentro de cada segmento. Esto es importante para saber el tamaño de cada grupo.
    - `.round(1)`: Redondea todos los valores numéricos resultantes de la agregación a un decimal, lo que mejora la legibilidad del informe.

- `segment_analysis.columns = ['_'.join(col).strip() for col in segment_analysis.columns.values]:`
    - Se utiliza `.agg()` con múltiples funciones para una misma columna (como `monetary`), Pandas crea nombres de columnas multinivel (por ejemplo, `('monetary', 'mean')).`
    - Esta línea de código es un list comprehension que itera sobre esos nombres de columnas y los une con un guion bajo (_) para crear nombres de columnas simples y planos,
      como `recency_mean, frequency_mean, monetary_mean` y `monetary_count`. Esto hace que el dataframe `segment_analysis` sea más fácil de manejar y leer.

- `display(rfm.head()):`
    - Esta línea simplemente muestra las primeras 5 filas del DataFrame rfm. Esto para recordar cómo se ve el DataFrame rfm después de todas las transformaciones y antes de mostrar el análisis final por segmento.

- `print('\n'), print("\033[1;4mAnalisis por Segmento\033[0m"), display(segment_analysis):`

- Estas líneas imprimen un salto de línea, luego un título en negrita y subrayado (Codigo ANSI) para la sección de análisis por segmento, y finalmente muestran el DataFrame “segment_analysis” con las estadísticas resumidas para cada grupo de clientes.

```
ordered_segment = ['Champions', 'Loyal Customers', 'Potential Loyalists',
'Lost Customers', 'At Risk']
```

Aquí solo se crea una lista para establecer un orden fijo para las próximas graficas debido que se quiere mantener un orden en las mismas y que no se auto ordenen de acuerdo al volumen.

```
plt.figure(figsize=(10, 6))
sns.barplot(x=segment_analysis.index, y=segment_analysis['recency_mean'],
palette='viridis', hue=segment_analysis.index, legend=False,
order=ordered_segment)
plt.title('Numero dias transcurrido desde la ultima compra por Segmento',
fontweight='bold')
plt.xlabel('Segmento')
plt.ylabel('Cantidad de dias')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Ahora en esta celda de código crea una gráfica de barras para visualizar el número promedio de días transcurridos desde la última compra (Recencia Media) para cada uno de tus segmentos de clientes. Así es como lo hace:

se Establece el tamaño de la figura. seguido se genera las barras se usa `sns.barplot` para dibujar las barras, donde:

- El eje X (`x`) muestra los nombres de los segmentos de clientes (`segment_analysis.index`).
- El eje Y (`y`) muestra el valor promedio de recencia (`recency_mean`) para cada segmento.
- `palette='viridis'` y `hue=segment_analysis.index` se usan para dar colores distintivos a cada barra.
- `order=ordered_segment` asegura que los segmentos se muestren en un orden específico (por ejemplo, de los más valiosos a los menos valiosos), usando la lista generada en la celda anterior.

Se Establece un título claro (`plt.title`), etiquetas para los ejes (`plt.xlabel, plt.ylabel`) y rota las etiquetas del eje X para mayor legibilidad. Por ultimo Muestra el Gráfico: Ajusta el diseño (`plt.tight_layout`()) y muestra la gráfica (`plt.show()`).

```
plt.figure(figsize=(10, 6))
sns.barplot(x=segment_analysis.index, y=segment_analysis['frequency_mean'],
palette='viridis', hue=segment_analysis.index, legend=False,
order=ordered_segment)
plt.title('Frecuencia Promedio de Compras por Segmento', fontweight='bold')
plt.xlabel('Segmento de Cliente')
plt.ylabel('Frecuencia Promedio de Compras')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Esta celta conserva la misma lógica solo que se grafica a partir de la columna `'frequency_mean'` y se cambia el título principal y los títulos de los ejes, pero el resto se mantiene igual a nivel de código. Pero tenemos la siguiente conclusión:

- Los clientes denominados Champions compran mucho más que los demás segmentos estos son los que dan flujo de caja

```
plt.figure(figsize=(10, 6))
sns.barplot(x=segment_analysis.index, y=segment_analysis['monetary_mean'],
palette='viridis', hue=segment_analysis.index, legend=False,
order=ordered_segment)
plt.title('Capacidad de Gasto (Monetario Promedio) por Segmento de Cliente',
fontweight='bold')
plt.xlabel('Segmento de Cliente')
plt.ylabel('Gasto Monetario Promedio')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Sucede lo mismo que la celda anterior solo que esta trabaja con la columna `'monetary_mean'` y se cambia los títulos de los ejes y el principal. Aquí tenemos la siguiente conclusión y recomendación:

- Conclusión: Los clientes denominados Champions pueden gastar en promedio 9678, mientras que un cliente en riesgo (At Risk) gasta solo 384
- Recomendación: El servicio al cliente para los "Champions" debe ser prioritario (asistencia dedicada, preventas exclusivas), ya que perder a uno de ellos equivale financieramente a perder a 4 clientes denominados "loyal Customers o 25 de alto riesgo

```
plt.figure(figsize=(10, 6))
sns.barplot(x=segment_analysis.index, y=segment_analysis['monetary_count'],
palette='viridis', hue=segment_analysis.index, legend=False,
order=ordered_segment)
plt.title('Numero de Clientes por Segmento', fontweight='bold')
plt.xlabel('Segmento de Cliente')
plt.ylabel('Numero de Clientes')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Aquí volvemos a repetir el código, pero trabajamos con la columna `'monetary_count'` y se cambia los títulos de los ejes y el principal. Aquí tenemos la siguiente conclusión y recomendación:

- **Conclusión:** El segmento más grande es Potential Loyalists(1455). Aquí se tiene una gran oportunidad de aumentar el volumen de clientes ya que mostraron interés inicial y solo necesita un empujón para ser leales
- **Recomendación:** Enfocar algún esfuerzo de marketing aquí para atraerlos posiblemente a que se pasen a loyal customers

```
plt.figure(figsize=(10, 7))
sns.heatmap(segment_analysis[['recency_mean','frequency_mean','monetary_mean']
], annot=True, cmap='crest', fmt='.1f')
plt.title('Promedio de Métricas RFM por Segmento de Cliente',
fontweight='bold')
plt.ylabel('Segmento de Cliente')
plt.show()
```

Esta grafica si es distinta a las otras anteriores debido que aquí trabajamos con un mapa de calor y unimos todas las columnas del dataframe `segment_analysis` es decir `'recency_mean','frequency_mean','monetary_mean'` donde se desprende la siguiente conclusión:

- **Conclusión**: Se puede decir que Los 'Champions' son, por mucho, los clientes más valiosos: Este segmento destaca claramente con la recencia más baja (compraron hace muy poco), la frecuencia de compra más alta y el gasto monetario promedio más elevado.

“Hasta aquí llega el análisis de RFM, pero como no podemos dejar unos datos sin analizar continuamos con otros análisis como el calculo del Teorema de Pareto, Top de productos devueltos y segmentos de clientes con mayor tasa de devolución, Acumulación Geográfica entre otros análisis que se estarán mostrando en las próximas entregas”